

Especificación del Proyecto: Desarrollo de un Analizador Léxico

1. Descripción General

El objetivo de este proyecto es diseñar un analizador léxico (lexer) para un lenguaje personalizado, que será utilizado en una aplicación creada por el estudiante. Este lexer transformará un archivo de texto que contenga un “programa” en una lista de tokens correspondientes a la gramática definida. La especificación del lenguaje será diseñada de manera informal utilizando ejemplos ilustrativos de código.

El proyecto incluye las siguientes fases:

- **Diseño de la especificación del lenguaje:** Definición de la sintaxis básica del lenguaje.
 - **Implementación del analizador léxico:** Creación de un lexer funcional utilizando una herramienta de generación de analizadores (como ANTLR, Flex, o alguna otra disponible).
 - **Pruebas y validación:** Verificación del correcto funcionamiento del lexer, incluyendo la identificación de errores léxicos.
-

2. Tareas del Proyecto

1. Especificación del Lenguaje Personalizado:

- Crear una descripción clara y concisa del lenguaje.
- Proveer ejemplos de código que ilustren la sintaxis de las instrucciones y patrones del lenguaje.
- Nombrar el lenguaje, asegurando que refleje su propósito o funcionalidad.

2. Especificación del Analizador Léxico:

- Diseñar un lexer que analice archivos de texto y extraiga tokens según la especificación del lenguaje.
- Incluir soporte para alguno de los siguientes elementos:
 - Identificadores (válidos e inválidos según las reglas del lenguaje).
 - Palabras reservadas.
 - Operadores.
 - Delimitadores (como ;, ,, {, }).
 - Números (enteros y/o flotantes según la especificación).

3. Implementación del Analizador Léxico:

- Desarrollar el lexer con las siguientes funcionalidades:

- **Identificación de Tokens:** Convertir las entradas válidas en una lista de tokens.
- **Manejo de Errores:** Detectar e informar errores relacionados con:
 - Identificadores no válidos.
 - Símbolos desconocidos.
- **Mensajes de Error Informativos:** Cada error debe incluir:
 - Número de línea y columna donde se detectó el error.
 - Descripción clara del problema.

4. Pruebas del Lexer:

- Probar el lexer con varios ejemplos que incluyan:
 - Programas válidos que cumplan con la especificación del lenguaje.
 - Casos que generen errores, para verificar la utilidad de los mensajes de error.

3. Funcionalidades del Lexer

- **Reglas Léxicas:**
El lexer debe implementar algunas de las siguientes reglas básicas:
 - Reconocer identificadores válidos (e.g., `variable1`, `my_function`).
 - Detectar identificadores inválidos (e.g., `123abc`, `@var`).
 - Diferenciar palabras reservadas (e.g., `if`, `else`, `while`) de identificadores.
 - Reconocer símbolos válidos del lenguaje (e.g., operadores, delimitadores).
 - Manejar números en formatos permitidos (enteros, decimales, etc.).
- **Manejo de Errores:**
 - Detectar símbolos no reconocidos (ej. `$`, `#`).
 - Proporcionar mensajes detallados con ubicación precisa del error.
- **Salida del Lexer:**
 - Para cada entrada válida: Generar una lista ordenada de tokens en formato legible.
 - Para entradas con errores: Mostrar los errores detectados y su localización.

4. Ejemplo de Salida del Lexer

Dada una entrada de programa:

```
if x > 10 then
  y = x * 2;
else
  z = y / 3;
```

El lexer debe generar:

```
1: (KEYWORD, "if")
2: (IDENTIFIER, "x")
3: (OPERATOR, ">")
4: (NUMBER, "10")
5: (KEYWORD, "then")
6: (IDENTIFIER, "y")
7: (OPERATOR, "=")
8: (IDENTIFIER, "x")
9: (OPERATOR, "*")
10: (NUMBER, "2")
11: (DELIMITER, ";")
12: (KEYWORD, "else")
13: (IDENTIFIER, "z")
14: (OPERATOR, "=")
15: (IDENTIFIER, "y")
16: (OPERATOR, "/")
17: (NUMBER, "3")
18: (DELIMITER, ";")
```

En caso de errores:

Error List:

Line 1, Column 4: Unknown symbol '\$' detected.

Line 2, Column 1: Invalid identifier '123abc'.
